

Kolej linowa - symulacja procesow i IPC

Nowe mechanizmy po konsultacjach (stabilność IPC)

- **Semafor kredytowy turystów (`sem_tourists`, 200 slotów):** generator bierze token przed forkiem, turysta oddaje przy wyjściu (`atexit`). Zapobiega lawinie procesów i zapychaniu kolejek bez żadnych sleepów.
- **Łagodne zamykanie peronu:** `employee1` po `PLAT_SHUTDOWN` budzi wszystkie semafony (bramki, limity, krzesła, wyjścia) i przez ~0.5 s drenuje kolejkę peronu, odsyłając spóźnionym `PLAT_SHUTDOWN` w trybie `IPC_NOWAIT`.
- **Nieblokujące sendy do kolejek (`IPC_NOWAIT`):** turysta rezygnuje, gdy kolejka biletowa/peronowa jest pełna zamiast się blokować; brak wiszących `msgsnd`.
- **Timeouty po stronie turystów:** oczekiwanie na odpowiedź peronu z limitem czasu (5 s) i zwrotem semaforów, żeby nie blokować strefy inside.
- **Porządki w logach:** ciche traktowanie `ENOMSG`/`EIDRM`/`EINVAL` w IPC, brak spamujących perrorów po zamknięciu.

Symulacja nadal odwzorowuje kolej krzeselkową z VIP-ami, dziećmi, rowerzystami, STOP/WZNOWIENIE itp., ale powyższe zmiany eliminują zwały i przecieki kolejek.

1. Srodowisko i narzedzia

- System operacyjny: Windows 11 + WSL2 (Ubuntu 24.04.3 LTS)
- Język: C11
- Kompilator: GCC (toolchain Ubuntu)
- Zarządzanie kompilacją: Makefile / GNU Make
- Edytor: VS Code (setup WSL)

2. Budowanie i uruchomienie

Budowanie:

```
make
```

Uruchomienie (argumenty: `Tp`, `Tk`, opcjonalnie `stop_once`):

```
./projekt <Tp> <Tk> [stop_once]
```

Przykład:

```
./projekt 0 10
```

Parametr `stop_once`: - 1 - po 5 sekundach symulacji wysyłany jest STOP i następnie WZNOWIENIE. - 0 lub brak - brak wymuszonego zatrzymania.

3. Pokrycie wymagań (skrot)

- Krzeselka 4-osobowe, 72 sztuki, max 36 zajętych naraz.
- Dopuszczalne składy: 2 rowerzystów, albo 1 rowerzystę + 2 pieszych, albo 4 pieszych.
- Losowe przyjsia, część turystów może zrezygnować.
- Sprzedaż biletów/karnetów w kasie.
- Karnety: jednorazowe, czasowe (Tk1/Tk2/Tk3), dzienne.
- Zniżka 25% dla wieku <10 i >65.
- Dzieci <8 pod opieką dorosłego; dorośli max 2 dzieci (4-8).
- Bramki dolne: 4 równoległe.
- Bramki peronu: 3 równoległe, otwierane przez pracownika1.
- Limit N osób w strefie między bramkami.
- Wyjście na górę: 2 równoległe pasy.
- Pracownik1 i Pracownik2, STOP/WZNOWIENIE sygnałami.
- Trasy zjazdowe T1 < T2 < T3.
- Godziny pracy Tp..Tk, po Tk brak wejść na bramkach, peron się opróżnia, po 3 s koniec.
- Logowanie użycia karnetu na bramkach.
- VIP (~1%) bez kolejki, ale z ważnym karnetem.
- Raport z liczby przejazdów na koniec.

4. Struktura kodu i odpowiedzialności

src/main.c

- Parsuje Tp/Tk (i stop_once), sprawdza zakres, ustawia czas startu.
- Tworzy kolejki, semafore (bramki 4/3, limit N, 36 krzeseł, 2 wyjścia, mutex shm, kredyt turystów 200) i pamięć dzieloną; publikuje je w ENV.
- Inicjalizuje stan kolei w shm (ring 72 krzeseł, liczniki, pid pracowników), czyści `report.txt`.
- Uruchamia kasjera, pracowników, generator turystów (spawn przez czas Tk-Tp z kontrolą `sem_tourists`).
- Po Tk: wysyła `PLAT_SHUTDOWN`, czeka na opróżnienie krzeseł, wysyła `shutdown` do kasy/peronu (z krótkimi timeoutami na ACK), ubija `employee2`, czeka na procesy, generuje raport, sprząta IPC.

src/cashier.c

- Słucha kolejki biletowej; każdy request ma pid, wiek, VIP, rowerzystę, liczbę biletów (w tym ulgowych).
- Losuje typ karnetu (lub honoruje podany), nalicza zniżkę 25% dla <10 i >65, ustawia `issued_at` i `valid_until` (Tk1/Tk2/Tk3/dzienny -> Tk).
- Po upływie Tk odrzuca sprzedaż (status != ST_OK).
- Odsyła wynik do pid turysty przez `msgrcv/msgrcv` z `mtype=pid`.

src/employee1.c

- Rejestruje PID w shm, czeka na PID emp2.
- Obsługuje kolejkę peronu (priorytet VIP): wpuszcza do Tk, po Tk odsyła PLAT_SHUTDOWN.
- Kolejkuje rowerzystów i pieszych, formuje składy (2 rowery / 1 rower+2 pieszych / 4 pieszych), rezerwuje miejsca w shm (limit 36).
- Sygnały STOP/WZNOWIENIE (SIGUSR1/2), tryb **stop_once** (pauza po 5 s, wznowienie po ~9 s).
- Na PLAT_SHUTDOWN: budzi wszystkie semaforey, flushuje oczekujących, drenuje kolejkę ~0.5 s w trybie IPC_NOWAIT i wysyła ACK.

src/employee2.c

- Rejestruje PID w shm, czeka na PID emp1.
- Sygnały STOP/WZNOWIENIE z handshake.
- Opróżnia krzesła z ringu (head++, zwalnia semafor krzesel); zatrzymuje się na sygnał/koniec.

src/tourist.c

- Pobiera kredyt z **sem_tourists** (przed forkiem), oddaje przy wyjściu (**atexit**).
- Losuje dzieci (max 2); wtedy wyłącza rower.
- Wysyła żądanie do kasy (IPC_NOWAIT — rezygnacja gdy pełna), czeka na odpowiedź.
- Etapy: dolne bramki (limit N + 4 bramki), peron (PLAT_REQ z timeoutem, priorytet VIP), górne wyjście (2 pasy). Loguje do **report.txt**.
- Jednorazowy przejazd (po zmianach: jeden zjazd i koniec procesu).

src/ipc.c

- Wrappery na IPC: kolejki (blokujące i IPC_NOWAIT), semaforey (wait/post), shm; helpery ENV; ciche ENOMSG/EIDRM/EINVAL po zamknięciu.

src/platform_queue.c

- Bufory oczekujących: rowerzyści osobno, piesi z rozmiarem grupy.
- Algorytm: 2 rowery -> 1 rower + 2 pieszych -> 4 pieszych. Rezerwacja miejsca (sem krzesel, mutex shm) i wysyłka PLAT_RES.

src/utils/tourist_utils.c

- Bramki dolne: weryfikuje ważność karnetu, rezerwuje limit N i gate4, loguje.
- Peron: PLAT_REQ (IPC_NOWAIT, timeout 5 s), w razie rezygnacji oddaje sem_inside; loguje przejście.

- Wyjście górne: `sem_exit2` (2 pasy), loguje; helpery `ENV/log/losowania` dzieci.

`src/utills/main_utills.c`

- Setenvy dla semaforów/liczb, cleanup IPC (kolejki, semy, shm).
- `generate_report`: zlicza wjazdy na peron i dopisuje podsumowanie.

`include/*.h`

- Struktury, stale, prototypy funkcji, klucze IPC i zmienne środowiskowe.

5. Jak to działa (opis mechaniki)

IPC

- Kolejki komunikatów: turysta $\leftrightarrow$ kasjer, turysta $\leftrightarrow$ peron.
- Semafore: limit N osób, bramki (4 i 3), krzeselka (36), wyjścia (2), dostęp do shm.
- Pamięć dzielona: stan kolei (ring krzeselek, liczniki, pid pracowników).

Kolejka i krzeselka

- Peron składa grupy w zgodzie z zasadami (piesi/rowerzysci, dzieci).
- Krzeselka trzymane w ringu o długości 72, z limitem 36 jednocześnie zajętych.

Czas i wygasanie karnetów

- `Tp` i `Tk` są parametrami uruchomienia.
- Karnety czasowe: `valid_until = issued_at + TkX`.
- Karnet dzienny: `valid_until = Tk`.
- Turysta jeździ w petli do przekroczenia `valid_until`.

STOP/WZNOWIENIE

- STOP (SIGUSR1) wysyłany do drugiego pracownika.
- Odpowiedź WZNOWIENIE (SIGUSR2) po potwierdzeniu gotowości.
- `stop_once` wymusza pojedyncze zatrzymanie po 5 sekundach.

6. Logi i raport

- `report.txt` zawiera log przejść przez bramki (`pass_id + czas + gate`).
- Na końcu pliku dopisywane jest podsumowanie liczby przejazdów na karnet (`gate=platform`).

7. Testy (manualne)

- Poprawne składy i limit 36 miejsc – kontrolowane w `platform_try_form_groups` + semafor krzeseł, zwalniany w `employee2`.
- Natychmiastowe zatrzymanie/wznowienie – sygnały SIGUSR1/2 między pracownikami, opcjonalny `stop_once`.
- VIP przed zwykłymi – peron odbiera z priorytetem VIP (mtime ujemny).
- Sprzątanie zasobów – `cleanup_ipc` usuwa kolejki/semafony/shm; po zakończeniu `ipcs` puste.

8. Funkcje wymagane przez projekt (gdzie szukać)

- Tworzenie procesów: `fork/execl` i `waitpid` w `process_utils.c`; pętlowy `spawn` w `spawn_processes_for_seconds_collect`.
- Sygnały: obsługa `kill/signal` w `employee1.c` i `employee2.c`.
- Kolejki komunikatów: `msgget`, `msgsnd`, `msgrcv`, `msgctl` w `ipc.c`.
- Semafony: `semget`, `semctl`, `semop` (wait/post) w `ipc.c`.
- Pamięć dzielona: `shmget`, `shmat`, `shmdt`, `shmctl` w `ipc.c`.
- Pliki/raporty: `fopen/fclose` w `tourist_utils.c`, `fprintf` w `main_utils.c`.